
Exercise Session 8

Reinforcement Learning

Sebastian Griesbach / Prof. Carlo D'Eramo

Summer semester 2026 / June 18th

In this exercise, you will implement a Deep Q Network and learn to use PyTorch.

Exercise 1

The exercise consists of the following files:

- **agent.py** This file contains the DQN agent which is learning, it follows the same agent pattern you already know. But its update function does not take a single transition but a *batch* of transitions instead. You need to modify this file.
- **dqn.py** This file contains the architecture of the actual Deep Q Network which we will use as our function approximation. You need to modify this file.
- **replay_buffer.py** This file contains a simple implementation of a replay buffer. A Memory that saves transitions and lets you sample batches from them. You don't need to modify this file but do take a look, there is not much going on there.
- **trainer.py** Very similar to the training loops you have seen in past exercises. This class runs the actual training process and the interaction with the environment. Take a look and see what is different from previous exercises, it's not much. You don't need to modify this file.
- **run_exercise_08.py** This is your program entry point. Here all the classes are instantiated, the hyperparameters are set and the training process gets called. You might need to modify this file.

Please take a look at all components and make sure you understand how they work together to create the algorithm. This is all the code that is needed to create a deep reinforcement learning algorithm. (If you don't count the libraries we use).

Exercise 2

Open **dqn.py** and implement the neural network as described in the comments. Refer to the PyTorch documentation to find out how to do so.

Exercise 3

Open **agent.py** and complete the **update** function of **DQNAgent** following the comments. Again refer to the PyTorch documentation on how to perform the specific steps. Also, take a look at the other parts of the code, especially the **_update_target_net** function and understand what it does. If you think you are done execute **run_exercise_08.py**. If everything works correctly you should see clear learning progress after the training. Although 50 training episodes are probably not enough to converge to a good policy that lands most of the time. Often it just overs above the ground at that point. When everything looks good increase the number of training episodes to 200 and see if the lander now lands correctly most of the time.

Exercise Bonus

We might went a bit overboard with the size of the neural network in **dqn.py**. How small can you make the network before it stops solving the task? What are your observations when you keep everything the same but just decrease the size of the network? Try to make this code work for other environments for example the CartPole-v1 environment.